

Uses Case

Projet 3 : chatbot immobilier

01_data_preparation.py

- Automatisation du téléchargement des DVF
- Automatisation de la création des df_annee
- Nettoyage et Isolation Forest
- Concaténation des données
- Récupération des données communales (population, localisation)

```
<class 'pandas.core.frame.DataFrame'>
Index: 4023044 entries, 0 to 4023067
Data columns (total 12 columns):
 #   Column                Dtype
---  -
 0   Valeur fonciere        int64
 1   Code postal            category
 2   Commune                category
 3   Code departement       category
 4   Code type local        int64
 5   Surface reelle bati    int64
 6   Surface terrain        int64
 7   Année                  int64
 8   Nb_hab_com             int64
 9   Dens_hab_com           float64
10  latitude_mairie        float64
11  longitude_mairie       float64
dtypes: category(3), float64(3), int64(6)
memory usage: 327.5 MB
```

01_data_preparation.py

```
def nettoyage(df):
    df = df[df['Nature mutation'] == 'Vente']
    df = df[['Date mutation', 'Valeur fonciere', 'Code postal', 'Commune', 'Code departement', 'Code type local', 'Surface reelle bati', 'Surface terrain']]
    df = df.dropna(subset=['Valeur fonciere', 'Code postal', 'Code type local', 'Surface reelle bati']).reset_index(drop=True)
    df = df.drop_duplicates()
    df = df[df['Code type local'].isin([1, 2])]

    df['Date mutation'] = pd.to_datetime(df['Date mutation'], format='%d/%m/%Y', errors='coerce')
    df['Année'] = df['Date mutation'].dt.year
    df['Valeur fonciere'] = df['Valeur fonciere'].astype(str).str.replace(',', '.').str.replace(r'\s+', '', regex=True).astype(float).round().astype(int)
    df['Surface reelle bati'] = df['Surface reelle bati'].astype(int)
    df['Surface terrain'] = df['Surface terrain'].fillna(0).astype(int)
    df['Code type local'] = df['Code type local'].astype(int)
    df.drop(columns=['Date mutation'], inplace=True)

    # Filtres prix et surface minimum
    df = df[(df['Valeur fonciere'] >= 30000) & (df['Surface reelle bati'] >= 15)]

    numeric_df = df[['Valeur fonciere', 'Surface reelle bati', 'Surface terrain']]

    iso_forest = IsolationForest(contamination=0.15, random_state=42)
    predictions = iso_forest.fit_predict(numeric_df)

    df['anomaly'] = predictions
    df_clean = df[predictions == 1]
    df_clean = df_clean.drop(columns=['anomaly'])
    df_clean = df_clean.reset_index(drop=True)
```

01_data_preparation.py

- Création du JSON par ville

Nb_total_ventes, Nb_ventes_maisons, Nb_ventes_appartements,

Prix_moy, Prix_med, Prix_min, Prix_max, Ecart_type_prix,

Prix_m2_moy, Prix_m2_med, Prix_m2_min, Prix_m2_max,

Surface_moy_interieur, Surface_med_interieur, Surface_moy_terrain,
Surface_med_terrain,

Prix_moy_maison, Prix_med_maison, Prix_m2_moy_maison,
Prix_m2_med_maison, Surface_moy_maison, Terrain_moy_maison,

Prix_moy_appartement, Prix_med_appartement, Prix_m2_moy_appartement,
Prix_m2_med_appartement, Surface_moy_appartement,

evolution_prix_m2, tendance, variation_annuelle_prix_m2,
annee_plus_forte_hausse_prix, annee_plus_forte_baisse_prix, periode_donnees,
evolution_ventes, variation_annuelle_ventes,

population, coordonnees, Segment Commune, Prix_med_groupe_pop,
Ecart_pct_prix_m2_vs_groupe, Quartile_prix_m2_dans_segment, evolution
population, variation annuelle population

```
dico[('ORLEANS', '45')]
```

```
{'Nb_total_ventes': {'2020': 1110,  
  '2021': 2439,  
  '2022': 2216,  
  '2023': 1829,  
  '2024': 1548},  
'Nb_ventes_maisons': {'2020': 277,  
  '2021': 553,  
  '2022': 425,  
  '2023': 401,  
  '2024': 321},  
'Nb_ventes_appartements': {'2020': 833,  
  '2021': 1886,  
  '2022': 1791,  
  '2023': 1428,  
  '2024': 1227},
```

02_vector_indexing.py

- Constitution d'une base de données (data.gouv, observatoire NAFU et publications de notaires)
- Automatisation du téléchargement
- Extraction du texte et découpage en morceaux
- Représentation par un vecteur d'embedding stocké dans un index FAISS
- Résumé des 5 résultats obtenus via un LLM seq2seq

03_qa_system.py

- Recherche du nom de la ville dans la question (Subtilités : nom ville en double, ignore les noms de communes “Moyen”, “Eu”,...)
- Récupération des données de la/les villes reconnues
- Récupération de l'année si possible sinon on prend l'année la plus récente présente dans les données de la commune
- Initialisation de BERT choisi → CATIE-AQ/QAmemberta

03_qa_system.py

- Classification de la question : général, prix (maison/appartement), évolution, ventes, population, surface
- Construction des contexte en fonction de la classification
- Fallback
- Réponse courte, phrase, contexte, score, temps

04_summarization.py

Textes bruts	PDFs
• Appel à un modèle de type seq2seq Hugging Face : <code>plguillou/t5-base-fr-sum-cnn-dm</code> • Contrôle de la longueur du résumé renvoyé	• Extraction du texte des PDFs avec la librairie PyMuPDF • Même pipeline que le texte brut

05_sentiment_analysis.py

- Modèle utilisé :
cardiffnlp/camembert-base-tweet-sentiment-fr
- Pipeline de type *text-classification*
- Longueur maximale de 2000 caractères
- Paramètre `top_k=None`

« fenêtrée orientée rue » VS « bruyant »

Response body

```
{
  "label": "neutral",
  "score": 0.9489956498146057,
  "details": [
    {
      "label": "neutral",
      "score": 0.949
    },
    {
      "label": "positive",
      "score": 0.0395
    },
    {
      "label": "negative",
      "score": 0.0116
    }
  ]
}
```

Response body

```
{
  "label": "negative",
  "score": 0.9723039865493774,
  "details": [
    {
      "label": "negative",
      "score": 0.9723
    },
    {
      "label": "neutral",
      "score": 0.0157
    },
    {
      "label": "positive",
      "score": 0.012
    }
  ]
}
```

06_api_deployment.py

- POST :
 - /ask : répond à la question avec le JSON / chatbot BERT
 - /ask_pdf : répond à la question avec les pdf
 - /summarize : résume un texte
 - /summarize_pdf : résume un pdf
 - /sentiment : analyse un texte et donne un score positif/négatif
 - /health : information comme JSON chargé ou non, chatbot prêt, ...

Docker

- **Objectif** : packager l'API et l'interface Streamlit dans un environnement stable et reproductible
- **Image Docker API** : installe les dépendances, embarque le code et les données, expose le port 8001, utilise un cache Hugging Face centralisé, et lance l'API avec Uvicorn
- **Orchestration avec Docker Compose** : permet de lancer l'API (port 8001) et Streamlit (port 8501) ensemble, avec un cache partagé, une communication interne entre services, et un démarrage ordonné (API avant interface).

Aller plus loin

- Rajouter la partie automatisation dans l'API pour l'application
- Exploitation des variables longitude/latitude
- Approfondir l'exploitation des données communales
- Tester davantage de modèles Hugging Face
- Améliorer l'indexation des chunks
- Améliorer la lecture des PDFs via un OCR et un VLM

07_streamlit_app.py

<<

SAFIR

Navigation

Accueil

Questions numériques

Analyse d'annonce

Recherche sémantique

Résumé de texte

Résumé de PDF

 **SAFIR, votre assistant immobilier**

SAFIR vous aide à exploiter l'information immobilière en un seul endroit.

- **Données DVF** : interrogez les ventes et indicateurs par commune (prix, volumes, tendances, surfaces, contexte démographique).
- **Annonces** : analysez le ton et le sentiment d'un texte d'annonce.
- **Documents** : recherchez des passages pertinents dans vos PDF, ou obtenez des **résumés** de texte ou de fichiers PDF.

Utilisez le menu à gauche ou les raccourcis ci-dessous pour commencer.

Accéder aux questions numériques

Accès rapide

Analyser une annonce

Résumer un texte

Rechercher des informations textuelles

Résumer un fichier PDF

Merci pour votre attention !
Des questions ?